

Wentworth Smart Academic Planner

Final Design Document and Project Report

Senior Project | Wentworth Institute of Technology

Project Team	Ibukunoluwa Folajimi & Ben N. Le
Advisor	Professor Salem Othman
System Type	Web-based academic planning and schedule recommendation system
Primary Technologies	Python, FastAPI, Streamlit, pdfplumber, python-docx, OpenAI API

Table of Content

1. Overview	4
2. Purpose.....	4
3. System Summary	4
3.1. Student Features	4
3.1.1. Transcript Processing	4
3.1.2. Degree Progress.....	5
3.1.3. Schedule Builder	5
3.1.4. Curriculum Planner.....	5
4. Software, Hardware, and Networking Components.....	5
4.1. Software	5
4.1.1. Primary Software Components	6
4.2. Networking.....	6
4.3. Hardware.....	6
5. Pre-existing Components	7
5.1. Frameworks	7
5.2. Libraries	7
5.3. External Services	7
6. Important Design Decisions.....	7
6.1. Separation of Frontend and Backend	7
6.2. REST API Design.....	8
6.3. Transcript Parser Design.....	8
6.4. AI as an Assistant	8
6.5. Configuration Management.....	8
7. System Diagrams	9
7.1. System Architecture Diagram.....	9
7.2. Use Case Diagram	10
7.3. Transcript Processing Workflow	11
8. Major Milestones	12
8.1. Initial Planning.....	12
8.2. Backend Development	12
8.3. Frontend Development.....	12
8.4. AI Integration	12

8.5. Testing.....	12
8.6. Documentation.....	12
9. Goals of the Project.....	13
9.1. Goals Successfully Achieved.....	13
9.2. Goals Partially Completed.....	13
9.3. Goals Not Completed	13
10. Unknowns and Challenges.....	13
11. Things We Wish We Knew Before Starting	14
12. Final Reflections	14
13. References.....	15
13.1. Documentation.....	15
13.2. Libraries	15
13.3. API Key	15
13.4. Deployment	15

1. Overview

The Wentworth Smart Academic Planner is a web-based academic advising and course planning system developed to simplify the academic planning process for Computer Science students at Wentworth Institute of Technology. Students often rely on unofficial transcripts, degree requirement documents, prerequisite charts, and academic advisors to determine what courses they should take each semester. This process can be time-consuming and prone to mistakes.

Our system consolidates these tasks into a single application that allows students to upload an unofficial transcript, automatically identify completed courses, calculate degree progress, recommend eligible courses, detect schedule conflicts, and generate AI-assisted semester-by-semester curriculum plans.

The project combines deterministic planning algorithms with artificial intelligence to provide recommendations while still allowing students to make their own scheduling decisions.

2. Purpose

The motivation behind this project came from observing how difficult academic planning can be, especially for students nearing graduation or transferring between programs. Degree audits require students to manually compare completed courses against curriculum requirements while simultaneously checking prerequisites and available course offerings.

Our goal was to create a system that reduces this complexity by automating the repetitive aspects of degree planning. Instead of spending significant time interpreting transcripts and curriculum sheets, students can upload an unofficial transcript and immediately receive an overview of their academic progress together with recommendations for future semesters.

3. System Summary

3.1. Student Features

Students can:

- Log into the application
- Upload an unofficial transcript (PDF or DOCX)
- Automatically detect completed courses
- View completed and remaining degree requirements
- Review eligible courses
- Build a semester schedule
- Detect scheduling conflicts
- Generate an AI-assisted curriculum map

3.1.1. Transcript Processing

The system extracts text from uploaded transcripts and reconstructs course information even when PDF formatting causes line wrapping or missing spaces.

Passing courses are identified while:

- withdrawn courses
- failed courses
- co-op courses
- courses in progress

are automatically excluded.

3.1.2. Degree Progress

Completed courses are compared against Computer Science curriculum requirements to determine:

- completed requirements
- remaining requirements
- elective requirements

3.1.3. Schedule Builder

Students can create semester schedules using available course sections. The application checks for overlapping meeting times before allowing schedules to be finalized.

3.1.4. Curriculum Planner

Using the student's completed coursework and remaining requirements, the AI module generates a recommended sequence of future semesters while respecting prerequisite constraints.

4. Software, Hardware, and Networking Components

Several software tools and libraries were required throughout the development of this project. Since the application is entirely web-based, no specialized hardware beyond standard personal computers was necessary.

4.1. Software

The project was primarily developed using Python because of its extensive ecosystem for web development, document processing, and artificial intelligence.

The frontend was implemented using Streamlit, which provided a rapid way to build an interactive web interface without requiring traditional HTML, CSS, or JavaScript development. FastAPI was selected as the backend framework because of its performance, clean architecture, automatic API documentation, and support for asynchronous operations.

Transcript parsing relies on two libraries. PDF files are processed using pdfplumber, while Microsoft Word transcripts are processed using python-docx. These libraries allow the application to extract textual information before reconstructing completed course records.

Visual Studio Code served as the primary integrated development environment. Git and GitHub were used for source control, version management, and collaboration between team members.

Artificial intelligence functionality was implemented using the OpenAI API to generate personalized semester-by-semester curriculum recommendations.

4.1.1. Primary Software Components

- Python
- Streamlit
- FastAPI
- pdfplumber
- python-docx
- OpenAI API
- Git
- GitHub
- Visual Studio Code

4.2. Networking

The frontend communicates with the backend using HTTP requests that exchange JSON data. The backend communicates with the OpenAI API whenever the curriculum planner is requested. Since the prototype was developed locally, networking requirements remained relatively simple. However, the application architecture supports future deployment to a cloud environment.

4.3. Hardware

Development was performed using Windows laptops running Python development environments.

The application requires only:

- Personal computer
- Internet connection (for AI functionality)
- Modern web browser

No additional servers or specialized hardware were required.

5. Pre-existing Components

Although the majority of the application was developed specifically for this project, several third-party libraries and frameworks were incorporated to reduce development time and improve reliability.

5.1. Frameworks

FastAPI provides routing, request handling, and REST API support. Streamlit provides rapid frontend development.

5.2. Libraries

- pdfplumber
- python-docx
- pandas
- requests
- pydantic
- python-dotenv

5.3. External Services

The OpenAI API was integrated for curriculum recommendation generation. Rather than attempting to build a large language model from scratch, the project leverages OpenAI's existing infrastructure while focusing development effort on transcript processing and academic planning. Using established libraries significantly reduced development time while improving maintainability and reliability.

6. Important Design Decisions

Several important architectural decisions influenced the overall quality and maintainability of the application.

6.1. Separation of Frontend and Backend

Instead of placing all application logic directly inside Streamlit, the project separates presentation from business logic.

Advantages include:

- easier testing
- modular development
- clearer responsibilities
- easier future deployment

6.2. REST API Design

Communication between frontend and backend occurs exclusively through REST API endpoints using JSON. This allows other applications to potentially reuse the backend services without modification.

6.3. Transcript Parser Design

Originally, transcript parsing relied heavily on fixed formatting assumptions. However, testing quickly revealed that PDF extraction frequently merged words, removed spaces, and wrapped course information across multiple lines. Rather than depending on formatting, the parser was redesigned to recognize patterns involving course codes, grades, and credit values. This decision greatly improved parser robustness.

6.4. AI as an Assistant

Rather than allowing AI to determine course eligibility, deterministic planning logic first evaluates all prerequisite requirements. Only after valid remaining courses have been identified does the AI produce semester recommendations. This prevents AI from recommending courses that violate prerequisite constraints.

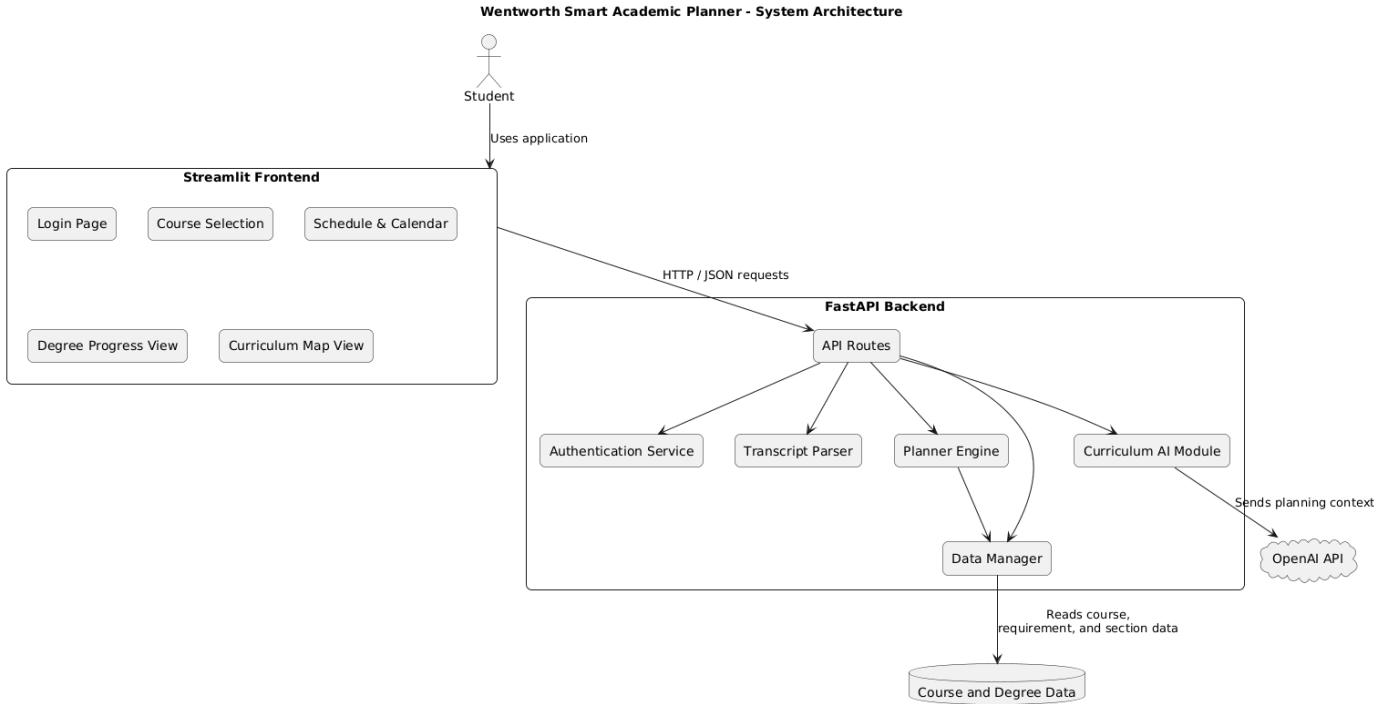
6.5. Configuration Management

Sensitive information such as API keys is stored in environment variables rather than being hardcoded into the application. This follows modern software engineering best practices and improves deployment flexibility.

7. System Diagrams

The following diagrams represent the major functional and technical aspects of the Wentworth Smart Academic Planner.

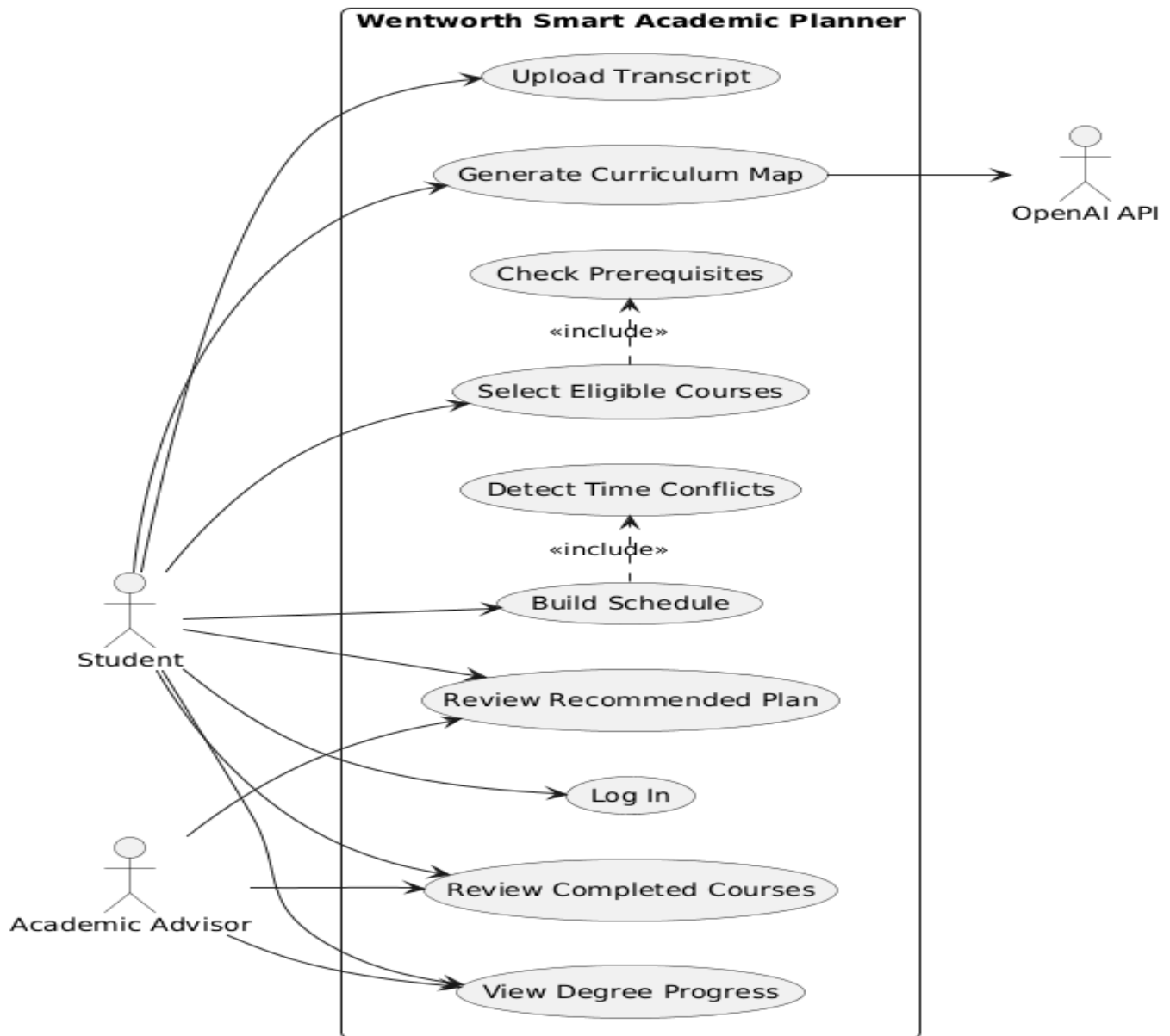
7.1. System Architecture Diagram



The System Architecture Diagram shows the overall organization of the application. It illustrates how users interact with the Streamlit frontend, how the frontend communicates with the FastAPI backend, and how the backend connects to the transcript parser, planning engine, project data, and OpenAI API.

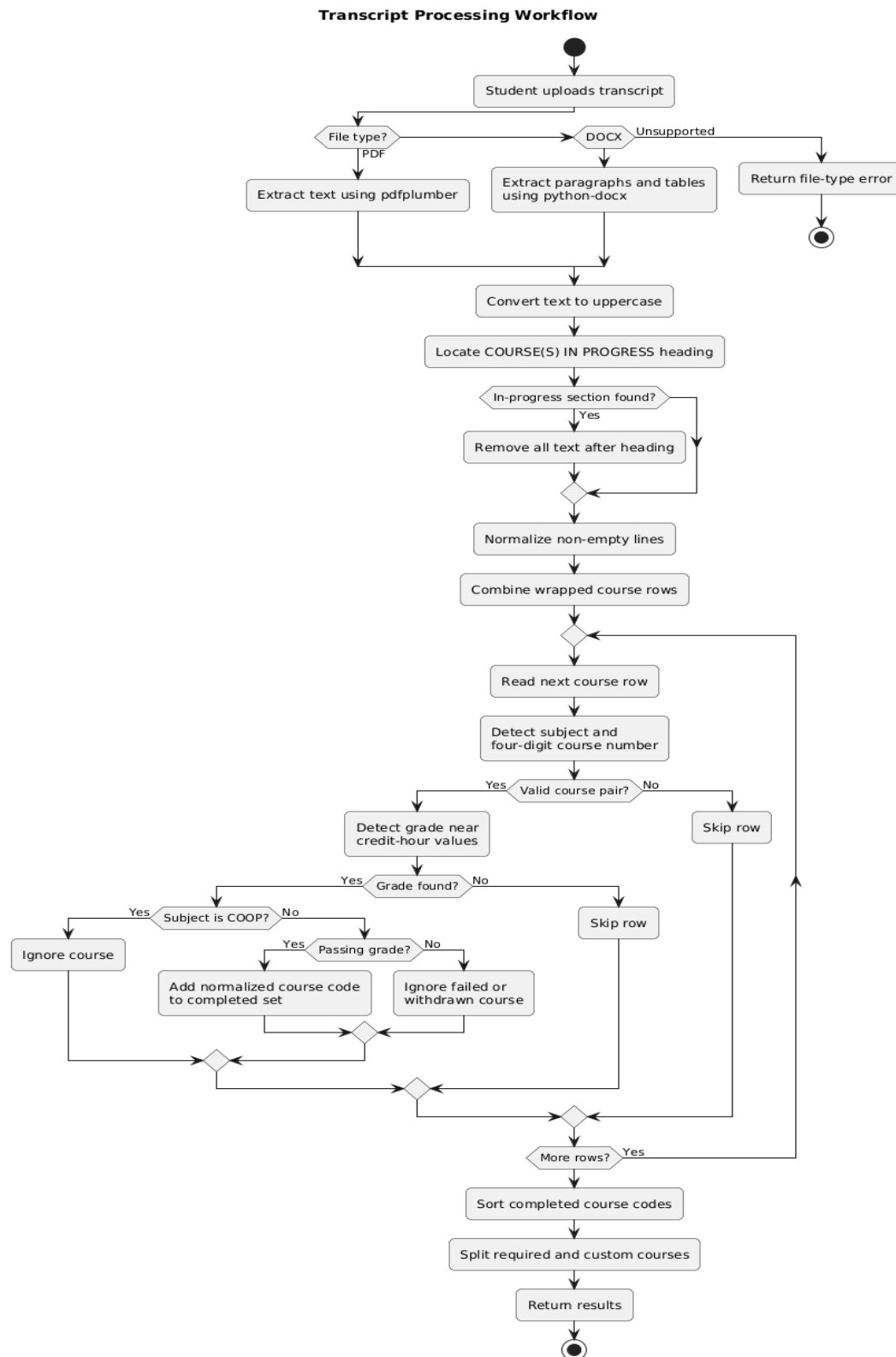
7.2. Use Case Diagram

Wentworth Smart Academic Planner - Use Case Diagram



The Use Case Diagram shows the main actions available to students. These include logging in, uploading a transcript, reviewing completed courses, viewing degree progress, selecting eligible courses, building a schedule, detecting time conflicts, and generating a curriculum map.

7.3. Transcript Processing Workflow



The Transcript Processing Workflow shows how an uploaded transcript is converted into a list of completed courses. It includes file validation, text extraction, removal of in-progress courses, reconstruction of wrapped course rows, grade detection, and exclusion of failed, withdrawn, and co-op courses.

8. Major Milestones

Development progressed incrementally throughout the semester.

Major milestones included:

8.1. Initial Planning

- Project proposal approved
- Technology stack selected
- Repository initialized
- Development responsibilities assigned

8.2. Backend Development

- FastAPI project initialized
- API routes implemented
- Transcript parser created
- Degree planner implemented

8.3. Frontend Development

- Streamlit interface completed
- Login interface created
- Course selection page implemented
- Schedule builder completed

8.4. AI Integration

- OpenAI API integrated
- Curriculum planning prompts refined
- AI recommendations tested

8.5. Testing

- Transcript parser tested using multiple unofficial transcripts
- Edge cases identified
- Schedule conflict detection validated
- Final debugging completed

8.6. Documentation

- Final report
- User manual
- Developer manual
- Testing report
- Final presentation

9. Goals of the Project

9.1. Goals Successfully Achieved

The major goals established during the proposal phase were successfully completed.

These include:

- Upload unofficial transcripts
- Parse PDF transcripts
- Parse DOCX transcripts
- Detect completed courses
- Ignore failed courses
- Ignore withdrawn courses
- Ignore co-op courses
- Ignore courses currently in progress
- Calculate remaining requirements
- Recommend eligible courses
- Detect scheduling conflicts
- Generate AI-assisted curriculum plans
- Provide a responsive web interface

9.2. Goals Partially Completed

Some goals require additional development before production deployment.

Examples include:

- Deployment beyond local development
- Support for every possible transcript format

9.3. Goals Not Completed

The following features were intentionally postponed:

- Integration with Wentworth's official registration system
- Automatic synchronization with Banner or DegreeWorks
- Student authentication using institutional login

10. Unknowns and Challenges

The greatest challenge involved transcript parsing. Although transcripts appeared visually identical, extracted text frequently differed dramatically.

Examples included:

- missing spaces
- merged course titles
- wrapped rows
- inconsistent section headings

Several iterations of parser improvements were required before reliable results were obtained. Another challenge involved balancing AI recommendations with deterministic prerequisite rules. Allowing unrestricted AI recommendations occasionally produced academically invalid schedules. This was solved by validating eligibility before sending planning information to the AI. Deployment also presented challenges due to environment configuration, API keys, and dependency management.

11. Things We Wish We Knew Before Starting

Looking back, several lessons would have significantly reduced development time. We underestimated how difficult PDF parsing would become. While reading a PDF appears straightforward, extracting reliable structured data proved to be one of the most time-consuming aspects of the project. We also wish we had more experience designing REST APIs before implementation. Several endpoints were refactored as the project evolved.

Prompt engineering required considerably more experimentation than anticipated. Producing useful curriculum recommendations involved numerous iterations before consistently generating practical semester plans.

Finally, we learned the importance of maintaining documentation throughout development rather than waiting until the end of the project. Updating documentation incrementally would have simplified final submission preparation and ensured that implementation details remained synchronized with the written design.

12. Final Reflections

Overall, the project successfully met its primary objectives. The final application provides an integrated environment for transcript analysis, degree planning, course selection, and AI-assisted curriculum planning.

The project also strengthened our experience in software engineering, API development, frontend/backend integration, document parsing, and AI-assisted application development.

Given additional time, we would expand the application by integrating with Wentworth's official academic systems, improving transcript compatibility, adding persistent databases, and supporting cloud deployment.

13. References

13.1. Documentation

- <https://fastapi.tiangolo.com/>
- <https://docs.streamlit.io/>
- <https://docs.python.org/>

13.2. Libraries

- <https://github.com/jsvine/pdfplumber>
- <https://python-docx.readthedocs.io/>

13.3. API Key

- <https://platform.openai.com/docs>

13.4. Deployment

- <https://render.com/>